

Project report

Authors: Halvor Thorsen, Mads Bolstad

Course: INF264

Summary:

This report is divided into three sections. In the first part, we talk about our approach to implementing the DecisionTree class and ID3 algorithm, and the reasons behind our choices. In section 2, we take a look at the performance of the different models, including plots and different findings, as well as model selection and a final evaluation. The final section takes a look at feature importance, before we have a conclusion and a couple of paragraphs about the use of AI tools and the division of labor.

Section 1

Section 1.1 - Class Design:

In decision_tree.py, we have defined a class DecisionTree. This contains a constructor and the methods that need to be imported in the other file, fit() and predict(). The DecisionTree constructor takes the arguments 'criterion' and 'max_depth', setting the default values "entropy" and None as specified by the project specification.

The class also contains a method print_tree(), made solely by ChatGPT, used to print the decision tree in a readable format. This method was used during early training for debugging purposes, but later dropped when we did a more complete model selection. What you see below is what the output of this method looked like, though shortened a bit. This output gave us a good understanding early in the process of how the decision trees actually worked.

```
Contract <= 0.696
  Yes:
    OnlineBackup <= 0.682
      Yes:
        PaperlessBilling <= 0.712
          Yes:
            OnlineSecurity <= 0.576
              Yes:
                Dependents <= 0.191
                  Yes:
                    DeviceProtection <= 0.368
                      Yes:
                        gender <= 0.581
                          Yes:
                            Partner <= 0.077
                              Yes:
                                InternetService <= 0.583
                                  Yes:
                                    tenure <= 1.600
                                      Yes:
                                        Predict: 1
                                      No:
                                        Predict: 0
                                  No:
                                    MultipleLines <= 1.429
                                      ...
                                      No:
                                        Predict: 1
                                      No:
                                        Predict: 0
```

Figure1: Shows printed tree

The `fit()` function is called for training the models. The method is based on the step-by-step process of the ID3 algorithm shown in the lecture about Decision Trees and the description in the project specification.

Step 1: The first step of the algorithm is to check if all datapoints have the same label. If that is the case, then there is no reason to split the tree further, and we return a node with that label. Considering all are the same in this scenario, we have gone for returning the first value for simplicity.

Step 2: If the first condition does not hold, then we move on to check if all datapoints have the same feature values. At this step, we also include a check if the tree is out of depth. If either of these conditions holds, then we find the most common label and return it.

Step 3: Now the algorithm has figured out it has to split the tree again. The ID3 algorithm splits on the feature that maximises information gain. It then adds a branch for each subset and recursively calls the algorithm on each of them. The tree is stored in `self.root`, built on nested dictionaries.

Predict() is called for model evaluation, done on the validation set. This takes in `self.root`, which is where our built tree is stored. The method traverses this tree until the node is no longer a dictionary. This implies we are at an end node, also called a leaf, and the method adds this leaf to a list of predictions. This list is what the method returns, and what we later use for evaluating the model's ability.

Computing the information gain

We compute the information gain in two separate ways depending on the criterion set for the selected model.

Entropy:

For models using entropy, information gain is computed as:

$$IG(x) = H(y) - H(y|x)$$

Where $H(y)$ is the entropy before the split, and $H(y|x)$ is the weighted entropy after splitting on x .

$$H(y) = - \sum_i P(y = y_i) * \log_2 P(y = y_i)$$

$$H(y|x) = \sum_i P(x = x_i) H(y|x = x_i)$$

$$= - \sum_i P(x = x_i) * \sum_j P(y = y_j | x = x_i) * \log_2 P(y = y_j | x = x_i)$$

Gini:

To compute Gini gain, we replace entropy with Gini impurity, but the structure is basically the same: impurity before the split minus weighted impurity after the split.

$$IG(x) = G(y) - G(y|x)$$

Where $G(y)$ is the impurity before the split, and $G(y|x)$ is the weighted impurity after splitting on x .

$$G(y) = \sum_j P(y = y_j)(1 - P(y = y_j))$$

$$G(y|x) = \sum_i P(x = x_i)G(y|x = x_i)$$

$$= \sum_i P(x = x_i) * \sum_j P(y = y_j|x = x_i)(1 - P(y = y_j|x = x_i))$$

Helper methods

This brings us over to the helper methods we made in `decision_tree.py`. We have three methods here which the `DecisionTree` class uses to compute information gain.

Decision_entropy() is the implementation of the formula for $H(y)$, while *gini()* is the implementation of the formula $G(y)$ written above.

Then we have the method *conditional_impurity()*, which takes in a feature and makes two subsets split on the mean of this feature. Then a simple if/else decides if we call on *decision_entropy()* or *gini()* for the weighted entropy/impurity, based on the criterion the method also takes as an argument.

These methods are called in *fit()* to compute the information gain. For steps 1 and 2 in the algorithm, we also made two helper methods so the code would be easier to read.

identical_labels() takes in the argument y and checks if every label is the same. The return value is binary.

identical_features() takes in the argument X and checks if all rows are identical. If they are, then that implies that all datapoints have identical feature values. This also has a binary return value.

Section 1.2 - Split threshold choice:

For every feature, the split point is the feature's mean. We also talked about using the median, or even a fancier solution trying several possible cut points, but ended with only one given threshold per feature. The project specification encouraged us to keep it simple and only use this, but we are aware that we may miss a better split point by not exploring other possible thresholds. Entropy and Gini can now only disagree on which feature to split on, but not where to split it.

Section 1.3 - Our approach:

We started with loading the dataset and looking at the data to get an understanding of what we were working with. We have not done any data processing or kept the small code for data exploration since it was specified that the dataset was preprocessed. When we started implementing the algorithm, we first implemented the first two steps before tackling information gain. It was at this point that we decided that helper methods would be useful, and ended up moving the code for the two primary steps outside the class. We started out with only implementing entropy, while gini was done at a later point.

When we (thought) we had implemented the *fit()* method correctly, we decided to test it out. At this point, we split up the dataset into training, validation, and test sets, and we made an instance of the model which we trained on the training set. This worked well, but we wanted to see the tree as well and therefore delegated the task of making the *print_tree()* method to ChatGPT. The reason for this was that it is not an actual part of the project, but rather just a tool we could use for clearer debugging.

The mistake we had made during training was the way we computed information gain. We had set a threshold on the actual decision entropy as well as the conditional entropy, meaning it only worked in practice for binary data. After fixing this and implementing *predict()*, we also began working on the gini measurement, which was the last missing piece in *decision_tree.py*.

Later, when computing permutation feature importance, we decided to put the method for computation in the *decision_tree.py* file so that the notebook would look cleaner.

Section 1.4 - Known limitations:

The way we build the tree is quite simple. We use a fixed threshold for the given feature to split on. Doing this instead of exploring other options likely has us missing a better split for some features, which would improve the model. Despite this, we still chose to keep it for simplicity. This also has implications for the difference between Entropy and Gini, as also stated in section 1.2. The two can differ on which feature to split on, but not where to split the features.

Section 2

Section 2.1 - Model selection

Dataset, training

The dataset churn-data.csv was ready for us from the start. The categories were already written as numbers, as well as no missing values, so there was no need to clean or change anything ourselves. The dataset has 7032 rows and 20 columns. 19 of the columns are features, and the last column is the target, Churn. A customer with the value 1 left the company, and a customer with the value 0 stayed as a customer. Three of the features are numbers (tenure, MonthlyCharges, and TotalCharges). The other 16 features are categories.

In run_experiment.ipynb, we load the file with np.genfromtxt, using the code to use. The first 19 columns become X, and the last column becomes y. The two groups are not equal in size, since 73,4% of the customers stayed while 26,6% left.

We split the data with train_test_split and used random_state = 42 to ensure that the split is the same every time. First, we use 80% of the data for training. The rest is then split in half into a validation set and a test set. This gives 5625 rows for training, 703 rows for validation, and 704 rows for testing. The share of customers who left is a little different in the three parts. It is 26,6% in the training set, 24,9% in the validation set, and 28,3% in the test set. The validation set has 175 customers who left, and the test set has 199.

How we picked the model:

We tested two hyperparameters. "Criterion" (entropy or gini) and max_depth (1, 2, 3, 4, 5, 6, 8, 10, 20, 40, and None). This gives 22 models, where every model is trained on the training set and scored on the validation set. All 22 models use the same validation set, so the comparison is fair.

We used the F1 score for the churn class to pick the best model, not accuracy. Most customers do not churn (75,1% of the validation set), so a model that always predicts "no churn" gets a high accuracy without finding any churn customers. We can see this in our results with max_depth=1, where the accuracy is 0.7511, but the F1 score is 0.0. This is because the model never predicts churn. F1 score catches this problem because it uses precision and recall for the churn class.

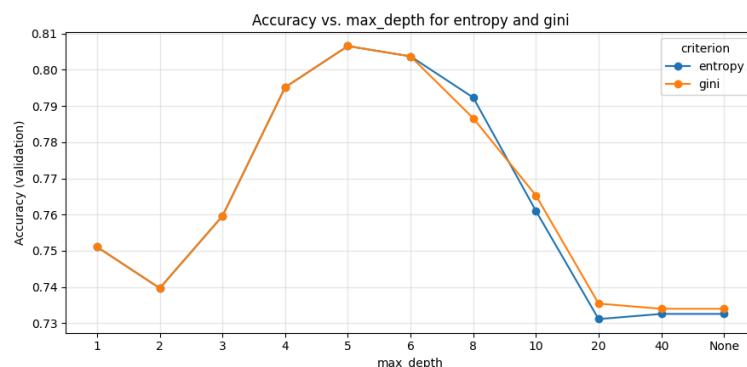


Figure 2: Shows accuracy vs max_depth for entropy and gini

We decided not to use k-fold cross validation. We used one validation set (hold-out). k-fold means training all 22 models k times, which takes longer, and we wanted to keep it simple. Our notebook runs in under a minute. The downside is that our validation set is small, with only 703 rows, and only 175 of them are churn customers. This means that small differences in F1 score between models can just be random variation. K-fold could give a more reliable result, because every row is used for validation once.

Results:

The best model on the validation set was with criterion = entropy and max_depth = 6, which gave an accuracy of 0.8037 and an F1 score of 0.5818. Gini with max_depth = 6 got exactly the same scores. Gini and entropy got the same scores for every max_depth from 1 to 6. They start to differ when max_depth = 8.

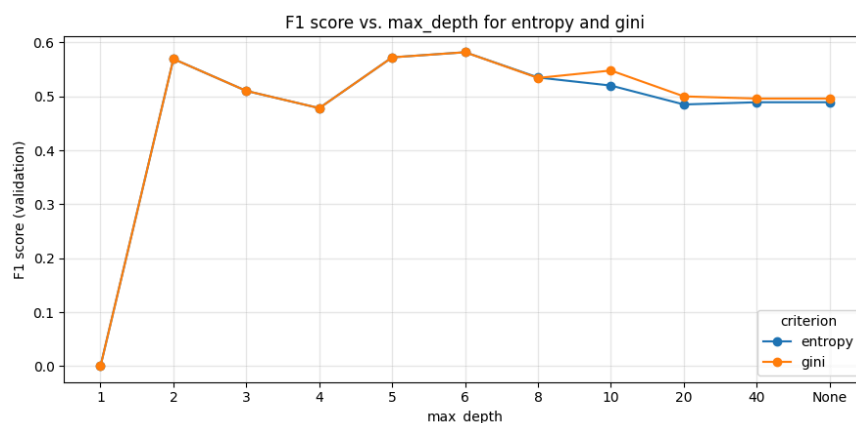


Figure 3: shows F1 score vs max_depth for entropy and gini

Discussion:

After running the notebook, we saw that entropy and gini got the same scores, so we needed a rule for choosing between them. The code keeps track of the best entropy model and the best gini model separately, then picks entropy if scores are equal. We chose entropy because it is the default criterion in our DecisionTree class and in the assignment text. Also, because ID3 was originally made with entropy. If two models with the same criterion get the same score, then the first one is kept. This is because we test the depth from small to large.

F1 score was a better choice than accuracy because of the class imbalance. With accuracy, the model with max_depth = 1 would look good even though it does not find any churn customers. Because the validation set is small, we should not put too much trust in small differences between models.

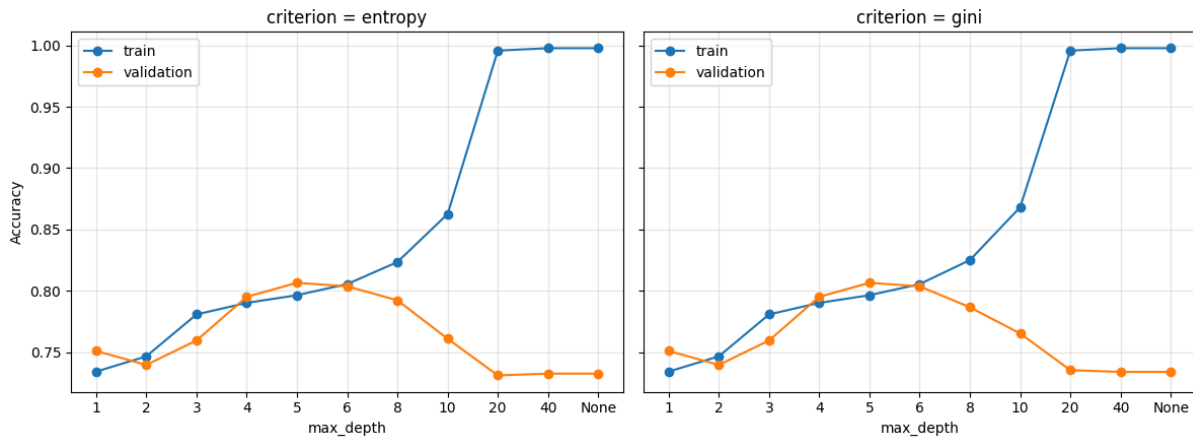


Figure 4: Shows difference in accuracy for each max depth

Section 2.2 – model evaluation and comparison to existing implementation

Models test performance:

We used the chosen model (entropy, with max_depth = 6), which was trained on the training set, and tested it on the test set. The test set was not used for anything before the model was chosen. The result for accuracy is 0.7642 and an F1 score of 0.5632. The confusion matrix on the test set is 431 true no-churn, 74 false churn, 92 missed churn, and 107 found churn. This gives a precision of 0.59 and a recall of 0.54 for the churn class.

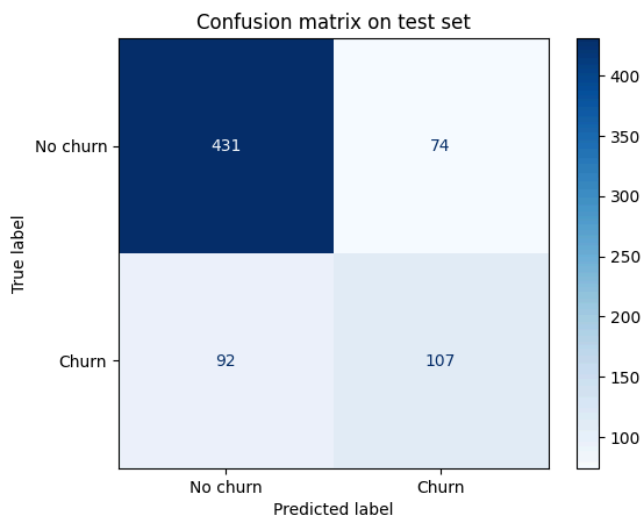


Figure 5: Confusion matrix for our model on the test set.

Both scores are lower than the validation set, with an accuracy of 0.8037 and F1 score of 0.5818. One reason is that we picked the model that was best on the validation set out of 22 models, so the validation score is a bit too good. Another reason is that the test set has a different share of churn customers (28.3%, against 24,9% in validation).

Scikit-learn comparison

We compared our model with DecisionTreeClassifier from scikit-learn. We did the same steps as for our own model with the same training, validation, and test sets. The same 22 combinations of criterion and max_depth, as well as the same rule for choosing the best model. We therefore chose the best F1 score on validation, as well as entropy if entropy and gini are equal. We used random_state = 42 here as well. We also did the model selection again for scikit-learn, and did not reuse our best settings from our best model.

The best scikit-learn model was criterion = entropy with max_depth = 2. Gini with max_depth = 2 got the exact same score, so entropy was chosen by our rule.

Discussion:

On the validation set, our model had a slightly higher F1 (0.5818) score than scikit-learn (0.5694). On the test set, scikit-learn had the higher F1 score (0.5813 against 0.5632), while our model had the higher accuracy of 0.7642 against scikit-learn 0.7074.

We think that the two models make different kinds of mistakes. The scikit-learn model is a small tree, with max_depth = 2. Therefore, it predicts churn more often. It predicts 293 of the 704 test customers, while only 199 are real churn customers. It finds more of the churn customers, but it also gives many false alarms. Its accuracy of 0.7074 is even lower than the 0.7173 we would get by always predicting “no churn”. Our tree is deeper with max_depth = 6, and predicts churn less often (only 181 customers). F1 score was our selection metric, so scikit-learn is slightly better by that measure. Which model is best in practice depends on what is more important. Either finding many churn customers, or avoiding false alarms.

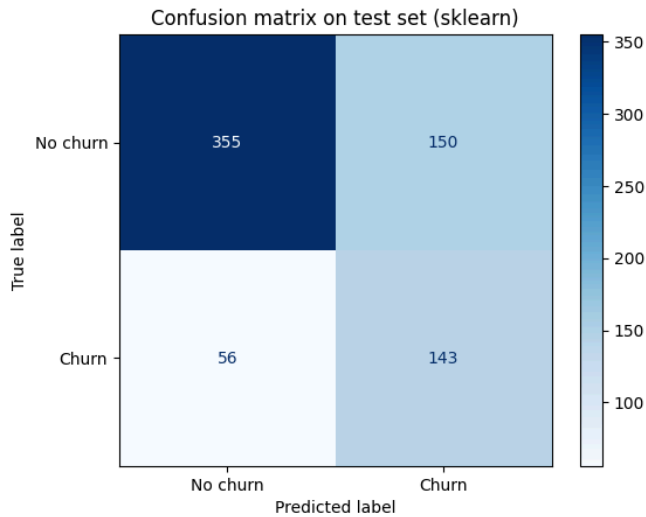


Figure 6: Confusion matrix for sklearn's model

	implementation	criterion	max_depth	accuracy	f1_score
0	Our model	entropy	6	0.7642	0.5632
1	sklearn model	entropy	2	0.7074	0.5813

Figure 7: Compared models on test set

We do not know if the F1 difference on the test set of 0.018 is real. The test set has only 199 churn customers, and the share of churn is different in the validation set and the test set, so the result could change with another split. K-fold cross validation would give a more reliable comparison. Since our F1 score (0.5632) is close to scikit-learn's F1 score (0.5813), therefore we think our implementation works as it should.

The trees can also be different because of how the split value is chosen. Our code splits at the mean of the feature in `decision_tree.py`. Scikit learn tries all possible split values and takes the best one.

Section 3 – Feature importance

Implementation:

The function was implemented in `decision_tree.py` with the required name and arguments

```
def permutation_importance(model, X, y, metric, n_repeats, seed):  
    np.random.seed(seed)  
  
    baseline_predictions = model.predict(X)  
    baseline_score = metric(y, baseline_predictions)  
  
    importances = []  
    n_features = X.shape[1]
```

Figure 8: permutation_importance implementation

The function first calculates the model accuracy on the original data. It then copies the data and shuffles one feature at a time. The model makes predictions on the shuffled data, and the selected metric is calculated. This is repeated for every feature and for every repetition. The importance score is the original accuracy minus the mean accuracy after shuffling the feature.

The model is not trained again after the shuffling. This means that the method measures how much the already trained model depends on each feature.

We used an entropy model with `max_depth = 6`. This was the best model from our model selection earlier in the report. The importance was calculated on the test set, which contains 704 observations. Accuracy was used as the metric. Each feature was shuffled 30 times, and the seed was set to 42, so that the results could be reproduced. The original test accuracy was 0.7642

Results:

The importance scores from our experiment:

gender	-0.0000
SeniorCitizen	0.0022
Partner	-0.0000
Dependents	-0.0000
tenure	0.0280
PhoneService	-0.0000
MultipleLines	-0.0083
InternetService	0.0226
OnlineSecurity	0.0305
OnlineBackup	0.0287
DeviceProtection	-0.0000
TechSupport	0.0066
StreamingTV	0.0008
StreamingMovies	-0.0000
Contract	0.0213
PaperlessBilling	0.0010
PaymentMethod	-0.0030
MonthlyCharges	-0.0000
TotalCharges	0.0033

Figure 9: Shows importance scores from our experiment

The bar plot in run_experiment.ipynb shows the same results:

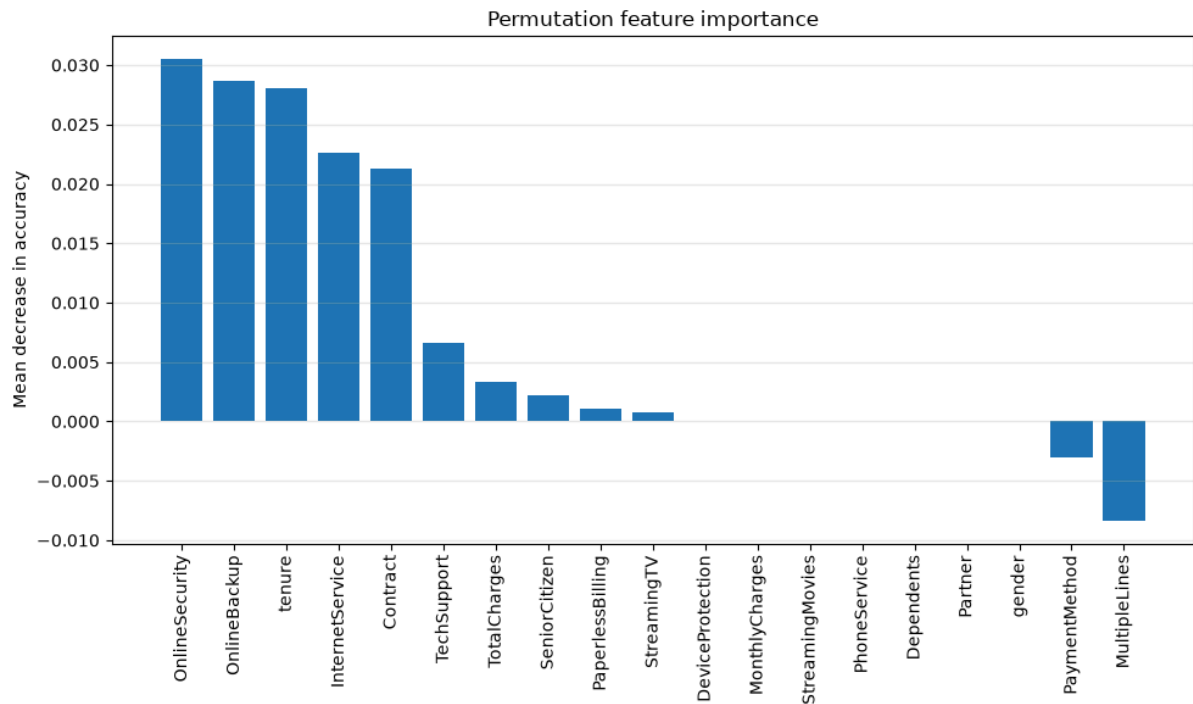


Figure 10: Permutation feature importance for our model

Here we can see that OnlineSecurity had the highest score of around 0.03, followed by OnlineBackup(0.029), tenure (0.028), InternetService (0.023), and Contract (0.021), which also have relatively high scores. Shuffling these features reduced the model accuracy more than shuffling other features. This suggests that the selected tree used these features more often or used them in more important splits.

Most of the other features had scores close to zero. Shuffling these features therefore had little effect on the predictions. MultipleLines (-0.008) and PaymentMethod (-0.003) had small negative scores. They do not show that these features improve the model when shuffled. It only means that some shuffled versions happened to give slightly higher accuracy than the original test data.

For the best scikit-learn model, we see that OnlineSecurity and Contract were the most important features. When we changed these features randomly, the accuracy fell by about 0.043 (4,3%) and 0.04 (4%). The other features had almost no effect.

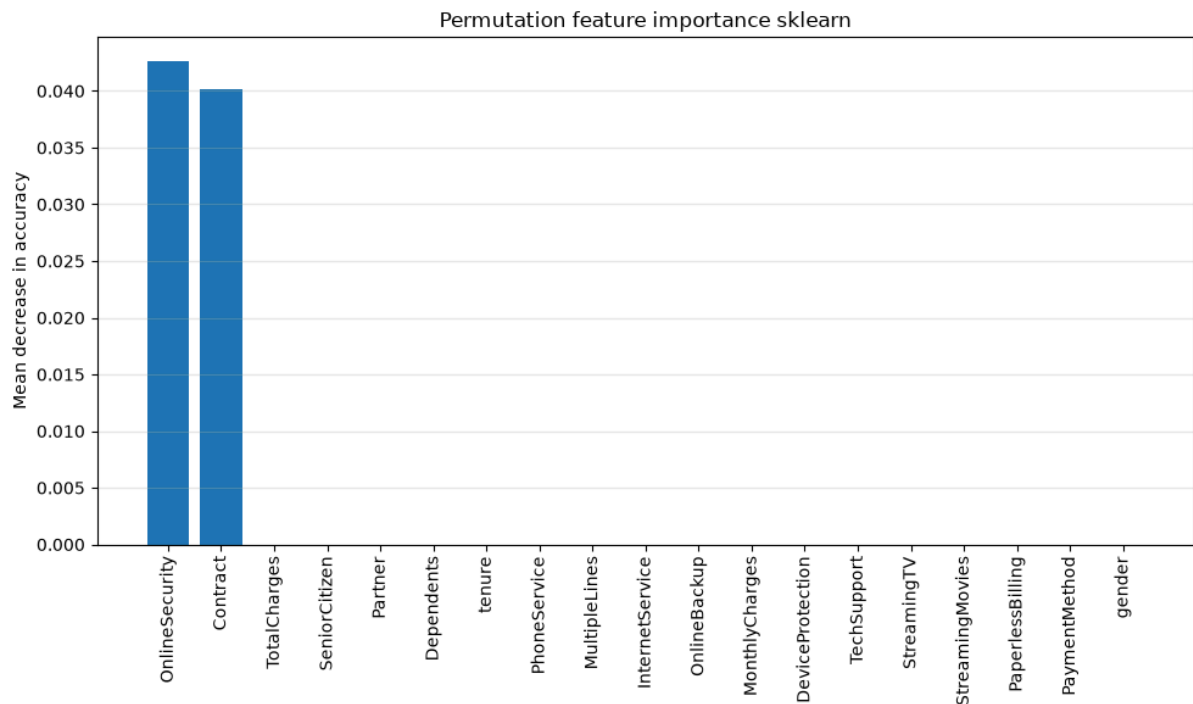


Figure 11: Permutation feature importance for sklearn's model

This is different from our own model, where more features were important. One reason may be that the scikit-learn model had a simpler tree with `max_depth = 2`.

Weaknesses of the method:

Permutation importance only shows how much the trained model relies on a feature. It does not show that the feature causes churn.

The results may change if we use a different decision tree, test set, random seed, or number of repetitions.

The results can also be affected by correlations between features. If two features contain similar information, the model may still perform well when only one of them is shuffled. This can make an important type of information appear less important.

We used accuracy, but the dataset is imbalanced. There are more customers who do not churn than customers who churn. Accuracy may therefore mainly reflect the majority class. Using F1 score or recall could give a different result.

Small negative and positive scores are uncertain. They may be caused by a random difference in the test set and the 30 shuffles.

Conclusion:

In this project, we implemented our own decision tree classifier using both entropy and Gini impurity and used it to predict which customers would leave a telecom company. We chose the best model based on a validation set with F1 score as the measurement, and tested the chosen model on the test set. Our best model used entropy as a criterion and a max depth of 6. It achieved an F1 score of 0.5632 and an accuracy of 0.7642 on the test set, which was close to the scikit-learn implementation at $F1=0.5813$ and $accuracy=0.7074$. These scores showed that our simpler approach to choosing split thresholds still produced decent results, although the scikit-learn model has a more advanced splitting procedure. We also used permutation feature importance to investigate which features influenced the predictions the most.

Use of AI tools:

We have used AI tools primarily as a helping source when we have been stuck developing or debugging the algorithm. All plots are made by AI. We did not want to spend much time making the plots, but rather focus on analyzing them and drawing conclusions ourselves. AI was also used to implement a method to print the built tree.

Division of labor:

We have met up and worked together for every part of this project. For the report, Section 1 was written by Mads while Sections 2 and 3 were written by Halvor. Coding has been done together throughout the whole project.

Sources:

scikit-learn developers. (2026). Permutation feature importance. In scikit-learn 1.9.1 documentation. Retrieved September 17, 2026, from https://scikit-learn.org/stable/modules/permutation_importance.html

Lecture slides from the INF264 course